

**Conception et developpement d'une
plateforme intelligente**
d'extraction d'informations a partir de documents
heterogenes

DocuAI / pfaEXTRACT

Realise par	[Nom et prenom]
Encadre par	[Nom de l'encadrant]
Etablissement	[Nom de l'etablissement]
Annee universitaire	2025-2026

Rapport etabli a partir de l'etat du depot au 15 juin 2026

Resume

Ce projet porte sur la conception et le developpement d'une plateforme intelligente capable de transformer des documents heterogenes en donnees JSON structurees et exploitables. La solution traite des images et des fichiers PDF appartenant a quatre familles principales : analyses medicales, factures STEG, tickets de caisse et factures fournisseurs. L'architecture actuelle associe une interface Next.js/React, une API FastAPI, des services Python de pretraitement et de routage, trois strategies d'extraction complementaires et une persistance SQLite/JSON.

La strategie retenue est hybride. Un OCR local specialise reste adapte aux documents relativement stables et explicables. Un pipeline IA local combine le pretraitement, Docling, un controle de qualite, PaddleOCR en secours et Qwen2.5 via Ollama. Gemini constitue une alternative cloud pour les mises en page plus variables. Les resultats sont normalises, valides, notes par completude, archives avec le document source et restitues dans une interface comprenant tableau de bord, historique, details, exports PDF/ZIP et page d'evaluation.

Le projet comprend egalement une chaine de preparation de donnees pour un futur fine-tuning LoRA de Qwen2.5. Le dataset prepare contient 1 199 exemples, repartis en 839 exemples d'entrainement, 178 de validation et 182 de test. Un smoke test controle a valide la detection de quatre familles sur quatre cas, tandis que les metriques globales Precision, Recall et F1 restent volontairement non annoncees tant que les predictions comparables des pipelines n'ont pas ete produites sur le split de test.

Mots-cles : OCR, extraction d'information, Document AI, Next.js, React, FastAPI, Docling, PaddleOCR, Qwen2.5, Ollama, Gemini, SQLite, LoRA.

Abstract

This project designs and implements an intelligent platform that converts heterogeneous documents into structured JSON data. The system supports medical laboratory reports, STEG utility bills, receipts and supplier invoices. Its current architecture combines a Next.js/React frontend, a FastAPI backend, Python preprocessing and routing services, three complementary extraction strategies, and SQLite/JSON persistence. The solution also prepares a supervised dataset for a future Qwen2.5 LoRA fine-tuning workflow. Measured functional results are reported separately from metrics that are still pending in order to preserve experimental integrity.

Sommaire

1. Introduction generale
2. Contexte, problematique et objectifs
3. Etat de l'art et choix technologiques
4. Methodologie et organisation du travail
5. Analyse des besoins et cas d'utilisation
6. Conception et architecture du systeme
7. Realisation technique
8. Preparation des donnees et fine-tuning
9. Tests, evaluation et resultats
10. Limites, securite et perspectives
11. Conclusion generale

12. Bibliographie et annexes

Liste des abreviations

Abreviation	Signification
API	Application Programming Interface
OCR	Optical Character Recognition
LLM	Large Language Model
VLM	Vision-Language Model
JSON	JavaScript Object Notation
JWT	JSON Web Token
SFT	Supervised Fine-Tuning
LoRA	Low-Rank Adaptation
CER	Character Error Rate
WER	Word Error Rate
PFA/PFE	Projet de fin d'annee / Projet de fin d'etudes

Chapitre 1 - Introduction generale

La transformation numerique produit un volume croissant de documents non structures : factures, tickets, analyses medicales, formulaires et pieces scannees. Ces documents contiennent des informations utiles, mais leur exploitation manuelle est lente, couteuse et sujette aux erreurs. Le projet DocuAI repond a ce probleme en proposant une chaine complete allant de l'import du document jusqu'a l'affichage, l'historisation et l'export des donnees extraites.

Le principal enjeu n'est pas seulement de lire du texte. Il faut reconnaitre le type du document, conserver sa structure, extraire les champs metier attendus, eviter l'invention de valeurs, signaler les incertitudes et rendre le resultat exploitable par une application. Pour cette raison, le projet compare et combine plusieurs pipelines plutot que de dependre d'un seul modele.

Question centrale

Comment automatiser l'extraction d'informations fiables depuis des images et PDF heterogenes, tout en conciliant precision, cout, confidentialite, explicabilite et simplicite d'utilisation ?

1.1 Contributions principales

- Une application web moderne en Next.js et React pour importer, configurer, visualiser et administrer les extractions.
- Une API FastAPI reutilisant les services Python historiques du projet.
- Un routeur heuristique capable d'orienter un document vers la famille STEG, medicale, ticket ou facture fournisseur.
- Un pretraitement d'image comprenant validation, correction d'orientation, rectification de perspective, deskew et amelioration visuelle.
- Trois strategies d'extraction : OCR local specialise, IA locale hybride et Gemini API.
- Des schemas JSON normalises, une validation metier, des warnings et un score de completude par famille.
- Une persistance double SQLite/JSON, incluant le document source, la corbeille et la generation de rapports PDF/ZIP.
- Un dataset train/validation/test et des scripts pour l'evaluation et un futur fine-tuning LoRA de Qwen2.5.

Chapitre 2 - Contexte, problematique et objectifs

2.1 Contexte documentaire

Les documents cibles varient fortement. Une facture STEG possede des zones relativement stables, alors qu'un ticket de caisse change selon l'enseigne, le pays et la qualite de l'impression. Une analyse medicale contient des tableaux, des unites et des intervalles de reference. Une facture fournisseur peut etre multilingue et presenter des lignes, taxes, remises et parties commerciales complexes. Cette diversite justifie une architecture modulaire et un routage adapte.

2.2 Problemes identifies

- Qualite variable des photos : perspective, rotation, ombres, flou, faible contraste.
- Documents PDF natifs ou scans ne proposant pas la meme qualite de texte.
- Bruit OCR, notamment sur les montants, dates, numeros et textes arabes.
- Mises en page heterogenes qui limitent les expressions regulieres fixes.
- Risque d'hallucination des modeles generatifs.
- Dependance possible a une API externe, avec cout, quota, latence et confidentialite.
- Temps de calcul important pour un modele local sur une machine sans GPU adapte.
- Necessite d'une trace technique pour expliquer les fallbacks et les erreurs.

2.3 Objectif general

L'objectif general est de construire une plateforme de bout en bout capable de recevoir un ou plusieurs documents, d'identifier leur nature, d'executer la meilleure strategie disponible, de produire un JSON metier valide et de conserver le resultat dans un historique consultable.

2.4 Objectifs specifiques

1. Supporter les formats PDF, JPG, JPEG, PNG, TIFF, WEBP et BMP jusqu'a 20 Mo.
2. Offrir un mode automatique et des modes forces par famille documentaire.
3. Permettre le choix entre moteur local, OCR classique et Gemini.
4. Produire des champs adaptes a chaque type de document.
5. Enregistrer les succes et les echecs pour faciliter l'audit.
6. Afficher des indicateurs d'activite, de statut et de completude.
7. Preparer une evaluation reproductible sur une verite terrain gelee.
8. Preparer les donnees necessaires a l'adaptation future de Qwen2.5.

Chapitre 3 - Etat de l'art et choix technologiques

3.1 OCR classique

L'OCR transforme une image en texte. Tesseract est pertinent pour une execution locale, gratuite et explicable. Il devient particulierement efficace lorsque le document possede une structure connue et que le pretraitement est adapte. En revanche, l'OCR brut ne comprend pas toujours la relation semantique entre une etiquette et sa valeur.

3.2 Vision par ordinateur

OpenCV est utilise avant l'OCR pour ameliorer la qualite d'entree. Les techniques employees comprennent la detection de contours, la recherche d'un quadrilatere, la transformation de perspective, l'analyse des lignes pour le deskew, le redimensionnement, le contraste et la nettete. Le pretraitement diminue le bruit transmis aux moteurs suivants.

3.3 Conversion documentaire structuree

Docling joue le role de convertisseur documentaire. Il cherche a conserver l'ordre de lecture, les blocs, les titres et les tableaux dans une representation Markdown. Il ne doit pas etre presente comme un simple modele OCR : sa valeur est la production d'un contenu structure plus facile a fournir a un LLM.

3.4 Modeles de langage et Vision-Language Models

Qwen2.5 est execute localement via Ollama. Le backend lui transmet le contenu du document, le type cible et un schema JSON strict. La temperature est fixee a zero afin de favoriser la stabilite. Gemini Vision constitue une reference cloud capable de lire directement les images et de produire une structure metier sur des documents complexes.

3.5 Justification des choix

Technologie	Role	Justification
Next.js 15 / React 19	Interface utilisateur	Composants reutilisables, routage App Router, experience web moderne.
TypeScript	Fiabilite frontend	Contrats de types pour les payloads API et reduction des erreurs.
Tailwind CSS	Design	Mise en page responsive, themes clair/sombre et composants coherents.
FastAPI	API REST	Validation, documentation automatique, upload multipart et integration Python.
Pydantic	Schemas	Normalisation et validation des structures medicales, tickets, STEG et fournisseurs.
OpenCV	Pretraitement	Correction geometrique et amelioration d'image.
Tesseract / PaddleOCR	Lecture de texte	Baseline locale et fallback multilingue.
Docling	Structure documentaire	Conversion de PDF/images en contenu structure.
Qwen2.5 / Ollama	Extraction locale	Confidentialite et absence de cout API par appel.
Gemini 2.5 Flash	Extraction cloud	Compréhension visuelle des documents variables.
SQLite + JSON	Persistence	Simplicité locale, audit lisible et portabilité.

Chapitre 4 - Methodologie et organisation du travail

4.1 Approche iterative

Le projet a ete construit par iterations courtes. Chaque iteration ajoute une capacite observable : extraction d'une famille, historisation, interface, API, pipeline local, authentication, evaluation puis preparation du fine-tuning. Cette organisation se rapproche de Scrum pour la planification incrementale et de pratiques XP pour le refactoring, les tests frequents et la livraison continue d'une version executable.

4.2 Demarche Data Science inspiree de CRISP-DM

1. Compréhension métier : identification des champs utiles pour chaque famille.
2. Compréhension des données : observation des images, PDF, annotations et erreurs OCR.
3. Préparation : nettoyage, normalisation, conversion des documents et création des splits.
4. Modélisation : règles OCR, prompts structurés, choix du pipeline et schémas Pydantic.
5. Évaluation : smoke tests, QA fonctionnelle, métriques champ par champ et analyse d'erreurs.
6. Déploiement applicatif : exposition FastAPI, interface Next.js, historique et exports.

4.3 Chronologie technique du travail

Phase	Travail realise	Livrable
1. Exploration	Collecte de documents et definition des champs.	Jeux d'images et schemas cibles.
2. Prototype OCR	Extraction medicale et STEG avec Tesseract/OpenCV.	Services Python specialises.
3. Integration Gemini	Prompts Vision et sorties JSON.	Pipelines cloud par famille.
4. Historique	Sauvegarde JSON puis SQLite, rapports PDF.	Audit et consultation.
5. Application web	Migration vers FastAPI + Next.js/React.	Interface admin multi-pages.
6. IA locale	Docling, PaddleOCR, Qwen2.5 et Ollama.	Pipeline hybride local.
7. Robustesse	Timeouts, fallbacks, warnings, corbeille, auth.	Application stabilisee.
8. Evaluation	QA, splits, métriques et scripts LoRA.	Protocole experimental reproductible.

4.4 Principe d'integrite experimentale

Regle de communication

Un score de completude affiche dans l'application n'est pas une accuracy scientifique. Les metriques Precision, Recall et F1 ne sont annoncees que lorsqu'une prediction brute est comparee a une verite terrain gelee.

Chapitre 5 - Analyse des besoins et cas d'utilisation

5.1 Acteurs

- Administrateur ou operateur : importe les documents, choisit une methode et consulte les resultats.
- Backend DocuAI : valide, route, traite, normalise et archive.
- Ollama/Qwen local : produit un JSON a partir du contenu structure.
- Gemini API : execute l'extraction cloud lorsque ce mode est selectionne.
- Base SQLite et systeme de fichiers : conservent les extractions et les sources.

5.2 Besoins fonctionnels

ID	Besoin	Implementation
BF01	S'authentifier ou creer un compte.	Login/register, token Bearer optionnel.
BF02	Importer un ou plusieurs documents.	Upload multipart et glisser-deposer.
BF03	Choisir auto, medical, STEG, fournisseur ou ticket.	Modes exposes par /api/meta.
BF04	Choisir local, OCR ou Gemini.	Parametre method du formulaire.
BF05	Visualiser le document et les champs.	Apercu image/PDF et composant ResultDetail.
BF06	Rechercher et filtrer l'historique.	Filtres type, texte, date et pagination.
BF07	Exporter un rapport ou plusieurs rapports.	PDF individuel et ZIP multi-selection.
BF08	Supprimer sans perte immediate.	Deplacement vers Data/history/trash.
BF09	Consulter les modeles et leur disponibilite.	Endpoint /api/models et detection runtime.
BF10	Consulter le protocole d'evaluation.	Page Evaluation IA et scripts de mesures.

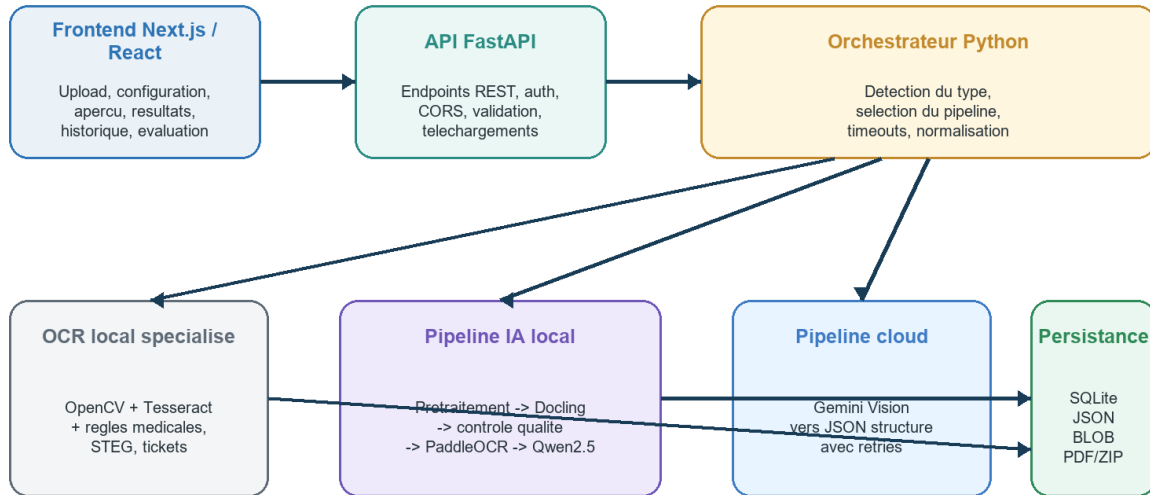
5.3 Besoins non fonctionnels

- Performance : limiter les blocages par des timeouts et des branches rapides.
- Robustesse : archiver aussi les echecs et produire des messages comprehensibles.
- Securite : ne pas committer les secrets, hacher les mots de passe et verifier les tokens.
- Confidentialite : proposer une execution locale sans transfert cloud.
- Maintainabilite : separer frontend, API, services, extraction, scripts et donnees.
- Traçabilité : conserver la methode, le type detecte, les warnings et la source.
- Ergonomie : interface responsive, themes et apercu direct des documents.

Chapitre 6 - Conception et architecture du systeme

6.1 Architecture generale

Architecture fonctionnelle de DocuAI



Flux principal : document -> extraction -> JSON valide -> historique exploitable

Figure 1 - Architecture fonctionnelle de DocuAI

L'architecture est organisee en couches. Le frontend ne connait pas les details des moteurs d'extraction : il consomme des contrats TypeScript correspondant aux payloads FastAPI. Le backend orchestre les services existants, tandis que les modules Python encapsulent le routage, le pretraitement, l'OCR, les appels IA, la validation et le stockage.

6.2 Flux principal

1. Le navigateur construit un FormData contenant les fichiers et la configuration.
2. FastAPI lit les fichiers et transfere le traitement lourd dans un threadpool.
3. Le document est copie dans un fichier temporaire et son type est detecte.
4. Le pipeline choisi produit un payload JSON normalise.
5. Le backend calcule warnings, resume et score de completude.
6. Le resultat et la source sont enregistres dans SQLite et dans un JSON lisible.
7. L'API retourne une cle d'historique et les URLs de detail, source et rapport.
8. Le frontend redirige vers la page Documents et affiche le resultat.

6.3 Detection automatique

Le routeur exploite d'abord le nom d'origine et le chemin logique. Cette precaution est importante car le backend travaille sur un nom temporaire aleatoire. Si aucun indice fiable n'est disponible, un texte leger est produit et plusieurs scores ponderes sont calcules. Les mots STEG, compteur et kWh favorisent la facture d'electricite; les termes laboratoire, hemoglobine ou TSH favorisent le medical; les mots caisse, paiement et merci favorisent le ticket; invoice, supplier, TVA, IBAN ou leurs equivalents arabes favorisent la facture fournisseur.

6.4 Schemas metier

Famille	Champs principaux
Analyse medicale	Laboratoire, medecin, patient, dossier, dates, tests, valeurs, unites, references, statut.
Facture STEG	Reference, compteur, date facture, montant, echeance, periode et coupon.
Ticket de caisse	Magasin, date, heure, numero, devise, articles, total et paiement.
Facture fournisseur	Numero, dates, devise, vendeur, client, articles, taxes, totaux et montant du.

6.5 Persistance

La table `extraction_history` contient l'identifiant, le type, le nom source, la date, le chemin relatif, le JSON, le statut, le message d'erreur, le type detecte, le MIME, la taille et le document source en BLOB. Un fichier JSON et une copie du document sont egalement ecrits dans un sous-dossier par type. Cette duplication facilite le debug et l'export, mais doit etre surveillee lorsque le volume augmente.

Chapitre 7 - Realisation technique

7.1 Frontend Next.js et React

Le frontend utilise l'App Router de Next.js avec des pages TypeScript. Les pages interactives sont des composants client et utilisent useState pour les formulaires, useEffect pour charger les donnees et useMemo pour les calculs derives. L'interface partage un AppShell responsable de la navigation, du theme et de la deconnexion.

Page	Responsabilite
/login et /register	Authentification, stockage du token et redirection.
/dashboard	Activite recente, graphiques et score qualite moyen.
/documents	Galerie, selection, detail, export PDF/ZIP et corbeille.
/extractions	Upload, apercu, choix du type et du moteur, lancement du batch.
/history	Historique groupe par date avec recherche et filtres.
/results	Affichage du dernier resultat ou d'une entree cible.
/evaluation	Dataset, metriques, pipelines et limites experimentales.
/settings	Theme, fournisseur IA, modele et configuration locale navigateur.

7.2 Communication avec l'API

Le module frontend/lib/api.ts centralise les appels fetch, l'ajout de l'en-tete Bearer, la lecture des erreurs FastAPI et les telechargements binaires. L'upload utilise AbortController avec une limite navigateur de 150 secondes. Les preferences de theme, methode et modele sont conservees dans localStorage; le token est conserve dans sessionStorage.

```
POST /api/extractions
FormData: files[], mode, method, geminiApiKey, geminiModel,
ollamaHost, localModel, retries, retryDelay, originsJson
```

7.3 Backend FastAPI

FastAPI expose les endpoints de sante, metadata, authentication, dashboard, modeles, historique, source, rapports et extraction. Les traitements synchrones lourds sont executes hors de la boucle asynchrone avec run_in_threadpool. Chaque document possede aussi une limite serveur configurable, fixee par default a 120 secondes.

Endpoint	Methode	Role
/api/health	GET	Etat du backend.
/api/auth/login	POST	Connexion.
/api/auth/register	POST	Creation d'un utilisateur.
/api/auth/me	GET	Utilisateur courant.
/api/meta	GET	Modes, moteurs et disponibilite runtime.
/api/dashboard	GET	Statistiques issues de l'historique.
/api/models	GET	Etat des moteurs et modeles.

Endpoint	Methode	Role
/api/history	GET	Recherche, filtres et pagination.
/api/history/{key}	GET/DELETE	Detail ou mise en corbeille.
/api/history/{key}/source	GET	Document source archive.
/api/history/{key}/report.pdf	GET	Rapport PDF individuel.
/api/history/export/zip	GET	Export groupe de rapports.
/api/extractions	POST	Traitement d'un batch.

7.4 Pretraitement des documents

- Validation de l'extension, de la taille maximale de 20 Mo et du contenu non vide.
- Correction de l'orientation EXIF.
- Detection du contour principal et transformation a quatre points.
- Estimation du skew par lignes de Hough.
- Amelioration du contraste et de la nettete.
- Redimensionnement raisonnable pour limiter le cout de calcul.
- Preparation des PDF et conservation d'une metadata detaillee des etapes.

7.5 Pipeline OCR local specialise

Ce pipeline associe Tesseract, OpenCV, expressions regulieres et heuristiques metier. Il est privilegie pour STEG et certaines analyses medicales. Des zones rapides peuvent etre lues pour la reference, le montant et la date. Le medical teste plusieurs variantes de niveaux de gris et de segmentation, puis choisit la meilleure selon un score interne.

7.6 Pipeline IA local hybride

Pipeline IA local hybride

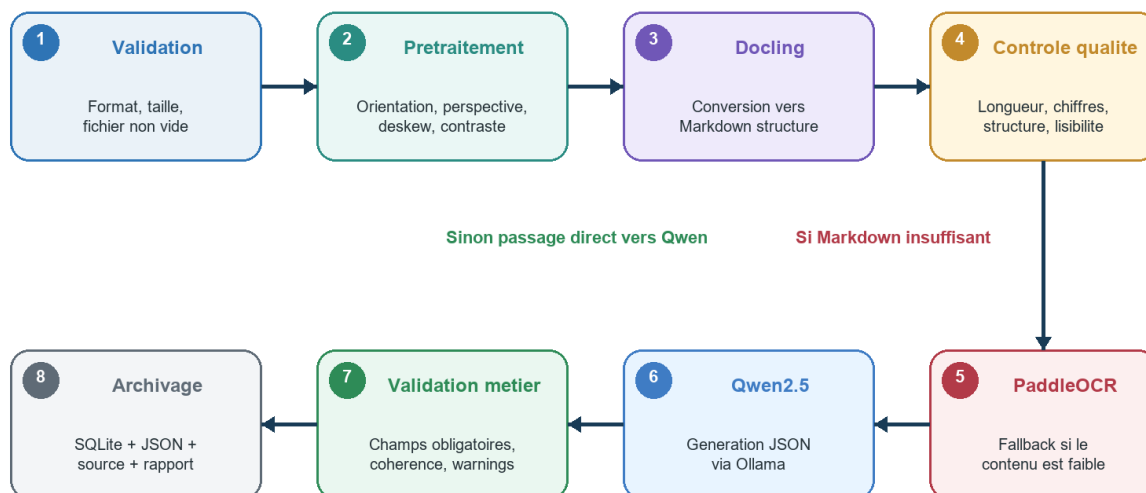


Figure 2 - Etapes du pipeline IA local

Pour les PDF, Docling est tente en premier. Si le Markdown est juge exploitable, il est envoye directement a Qwen. Sinon, PaddleOCR est lance avec une limite courte; un OCR Tesseract rapide peut encore prendre le relais. Le prompt impose un schema par famille, interdit l'invention et demande uniquement un JSON valide. Ollama est appele avec temperature zero, un contexte configurable et une limite de generation.

Detail important

Le mode nomme « local » ne signifie pas que Qwen est toujours utilise. Pour eviter les timeouts, les images STEG et certaines images medicales suivent des branches OCR rapides. Le champ local_pipeline indique la source reelle et, dans plusieurs fallbacks, qwen_used=false.

7.7 Pipeline Gemini

Gemini recoit l'image et un prompt specialise. Les pipelines normalisent ensuite la reponse selon les schemas Pydantic. La cle peut provenir du fichier .env ou etre fournie pour la session. Ce moteur offre une bonne comprehension visuelle, mais depend du reseau, du quota, de la politique de confidentialite et du cout du fournisseur.

7.8 Validation et score qualite

Le backend verifie les champs obligatoires, la presence de lignes et la forme des montants. Il ajoute des warnings lorsque des informations sont absentes ou ambiguës. Un score de completude pondere est calcule differemment selon la famille. Par exemple, la reference et le montant STEG ont un poids eleve; le medical valorise les identifiants, les dates et les lignes d'analyse; le ticket valorise le magasin, la date, le total, l'adresse et le numero. Ce score sert au tri et a l'interface, pas a mesurer une accuracy.

7.9 Authentification et securite

L'authentification est optionnelle et desactivee par default pour la demonstration locale. Lorsqu'elle est active, les mots de passe sont hashes par PBKDF2-SHA256 avec 210 000 iterations et un sel aleatoire. Le backend emet un token signe en HMAC-SHA256 contenant le sujet, la date d'emission et l'expiration. Les comptes crees sont stockes dans une base SQLite distincte. Les secrets doivent rester dans .env.

7.10 Rapports et historique

ReportLab genere un rapport PDF adapte au type de document. Le frontend peut telecharger un PDF individuel ou demander un ZIP contenant plusieurs rapports. La suppression deplace le JSON et la source vers une corbeille avant de retirer la ligne active de la base, ce qui evite une perte immediate.

Chapitre 8 - Preparation des donnees et fine-tuning

8.1 Objectif

Le fine-tuning n'est pas execute pendant chaque extraction. Il constitue une phase hors ligne destinee a specialiser Qwen2.5. Une fois un adaptateur LoRA entraine et integre dans Ollama, l'application pourra remplacer le nom du modele sans modifier l'architecture generale.

Repartition du dataset prepare

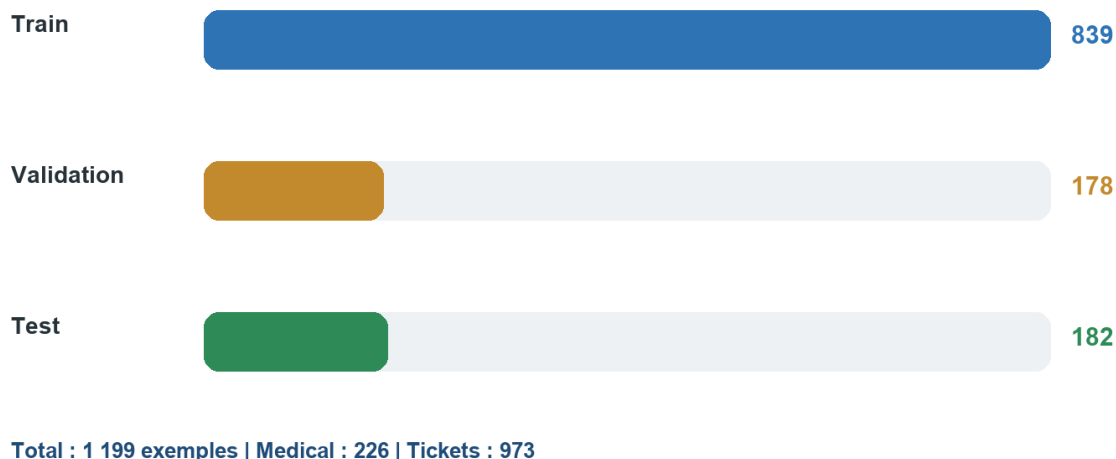


Figure 3 - Splits du dataset prepare

8.2 Composition des donnees

Split	Total	Medical	Tickets
Train	839	158	681
Validation	178	33	145
Test	182	35	147
Total	1 199	226	973

8.3 Format d'apprentissage

Chaque exemple contient une instruction, un texte d'entree, une sortie JSON de verite terrain et des metadata. Le split est stratifie par famille avec une graine fixe de 42, ce qui permet de reproduire la repartition 70 % / 15 % / 15 %.

```
{"instruction": "Extrais les informations importantes...",  
"input": "[METHOD_USED=...]\n\n<TEXTE DU DOCUMENT>",  
"output": {"document_type": "receipt", "...": "..."},  
"metadata": {"id": "...", "document_family": "receipt"}}
```

8.4 Chaine de preparation

1. Collecter ou generer les sources et leurs annotations.
2. Produire un texte d'entree OCR ou structure.

3. Normaliser la verite terrain vers le schema DocuAI.
4. Eliminer les exemples invalides ou trop ambigus.
5. Construire les splits sans melanger train et test.
6. Exporter le format interne ou le format conversationnel LLaMA-Factory.
7. Lancer un SFT LoRA sur une machine disposant des ressources necessaires.
8. Evaluer le modele adapte sur le split test gele avant integration.

8.5 Etat actuel

Etat reel au 15 juin 2026

Le dataset et les scripts de preparation, d'export et d'entrainement sont presents. Qwen2.5:7b-instruct est le modele local de base. Aucun modele fine-tune ne doit encore etre presente comme integre ou valide dans l'application.

Chapitre 9 - Tests, evaluation et resultats

9.1 Strategie de test

- Tests unitaires pytest pour les indices de routage et les fallbacks.
- Smoke tests multi-familles pour verifier la detection.
- QA d'integration par cas avec attentes sur les champs et les erreurs.
- Script quantitatif pour comparer les JSON predicts a la verite terrain.
- Inspection manuelle des documents difficiles et des warnings.

9.2 Resultats effectivement verifies

Verification	Date	Resultat	Interpretation
Detection STEG, medical, ticket, fournisseur	09/06/2026	4/4 cas corrects	Smoke test controle, pas benchmark global.
QA fonctionnelle recente	09/06/2026	7/7 cas passes	Verifie flux et fallbacks; ne prouve pas la precision globale.
Tests pytest du routeur	15/06/2026	2/2 passes	Nom d'origine conserve et fallback STEG verifie.
Split quantitatif	09/06/2026	182 documents	35 medical + 147 tickets.
Predictions comparables	15/06/2026	Aucune disponible	Precision, Recall et F1 globaux non calculables.

9.3 Metriques retenues

Metrique	Formule / definition	Usage
Detection Accuracy	Documents bien classes / documents testes	Qualite du routeur.
Valid JSON Rate	JSON lisibles / sorties produites	Exploitabilite applicative.
Field Accuracy	Champs attendus corrects / champs attendus	Qualite metier intuitive.
Precision	$TP / (TP + FP)$	Part des valeurs produites qui sont correctes.
Recall	$TP / (TP + FN)$	Part des valeurs attendues retrouvees.
F1-score	$2 \times Precision \times Recall / (Precision + Recall)$	Equilibre precision/rappel.
CER / WER	Distance d'edition / taille de reference	Qualite du texte OCR.
Latence	Temps moyen par document	Experience utilisateur et cout machine.

9.4 Analyse des echecs observes

Les journaux de QA montrent plusieurs modes d'echec utiles pour l'amelioration. Une facture STEG a produit un montant OCR incorrect dans un run anterieur, alors que la branche locale rapide a retrouve la valeur attendue. Une facture fournisseur locale a echoue lorsque Ollama n'etait pas joignable. Les tickets et factures fournisseurs traitees par fallback OCR peuvent produire un JSON exploitable mais avec un score faible et des champs a verifier. Ces cas confirment l'interet des warnings, de l'archivage des erreurs et de la comparaison par famille.

9.5 Precautions pour le benchmark

- Utiliser exactement les memes documents pour chaque pipeline.
- Geler le split test et ne pas regler les prompts sur ce split.
- Sauvegarder les predictions sans correction humaine.
- Ne pas melanger les exemples synthetiques avec un benchmark clinique reel.
- Ne pas compter le score de completude de l'interface comme une accuracy.
- Le raccourci receipt_test renvoie l'annotation SROIE lorsque le nom correspond au dataset; il doit etre exclu de toute mesure de modele.

9.6 Decision technique par famille

Type	Pipeline recommande	Raison
Facture STEG	OCR specialise; Gemini si image tres difficile	Structure assez stable, rapidite et explicabilite.
Analyse medicale	OCR structure + validation; IA en secours	Champs critiques et besoin de controle.
Ticket de caisse	Gemini ou IA locale hybride	Mise en page et contenu tres variables.
Facture fournisseur	Gemini ou Docling + Qwen	Semantique, multilingue, lignes et taxes.
PDF propre	Docling + Qwen local	Bonne conservation de la structure.

Chapitre 10 - Limites, securite et perspectives

10.1 Limites fonctionnelles et scientifiques

- Les metriques F1 ne sont pas encore produites pour les pipelines sur le split test.
- La verite terrain couvre principalement medical et tickets; STEG et fournisseur necessitent plus d'annotations.
- Les donnees medicales synthetiques ne remplacent pas une validation clinique anonymisee.
- Qwen2.5 local n'est pas encore fine-tune ni valide face a Gemini sur un benchmark commun.
- La qualite arabe reste dependante des langues OCR installees et de la resolution.
- Le batch est traite sequentiellement et aucune file de taches persistante n'est encore utilisee.
- Le stockage simultane en BLOB et sur disque augmente l'espace consomme.

10.2 Limites de l'interface actuelle

La page Dashboard contient encore des constantes de presentation et une mise a l'echelle statique pour certaines cartes et graphiques. Les statistiques backend existent, mais ces valeurs de demonstration doivent etre remplacees par le payload reel avant une livraison de production ou une presentation comme resultat mesure.

10.3 Durcissement securite

- Activer DOCUAI_AUTH_ENABLED et fournir un secret de token long et aleatoire.
- Utiliser uniquement le hash PBKDF2, jamais un mot de passe en clair en production.
- Deplacer le token frontend vers un cookie HttpOnly si l'application devient publique.
- Limiter CORS aux domaines deployes et activer HTTPS.
- Ajouter une autorisation par role et une politique de conservation des documents.
- Chiffrer ou externaliser les documents sensibles.
- Auditer les dependances et limiter les informations techniques exposees dans les erreurs.

10.4 Perspectives

1. Generer les predictions des trois pipelines sur le split test et calculer les metriques.
2. Annoter un benchmark STEG et fournisseur plus large.
3. Entraîner et evaluer un adaptateur LoRA Qwen2.5.
4. Ajouter Celery/RQ ou une file de taches pour les traitements longs.
5. Passer a PostgreSQL et a un stockage objet lorsque le volume augmente.
6. Ajouter une validation humaine et une boucle de correction pour enrichir le dataset.
7. Mettre en place Docker, CI/CD, tests end-to-end et monitoring.
8. Ajouter CER/WER et latence par etape dans le tableau d'evaluation.
9. Remplacer toutes les valeurs de demonstration du dashboard par des donnees reelles.

Chapitre 11 - Conclusion generale

DocuAI constitue une plateforme fonctionnelle et modulaire d'extraction documentaire. Le projet depasse un simple script OCR : il couvre l'import, la detection, le pretraitement, plusieurs moteurs, la normalisation, la validation, l'historisation, l'authentification, l'interface et l'evaluation. L'association Next.js/React et FastAPI offre une architecture claire, tandis que les services Python permettent de reutiliser les connaissances metier developpees pour chaque famille.

Le choix le plus important est l'approche hybride. L'OCR specialise reste pertinent lorsque la structure est stable; Docling, PaddleOCR et Qwen2.5 offrent une solution locale pour les documents variables; Gemini fournit une reference cloud. Le systeme enregistre la methode reelle et les fallbacks afin de ne pas masquer la complexite du traitement.

Enfin, le projet adopte une communication experimentale prudente. Les resultats fonctionnels disponibles sont documentes, mais aucun score global n'est invente. La prochaine etape decisive est la production de predictions comparables sur le split test, puis l'entrainement et l'evaluation d'un Qwen specialise. Cette base rend le projet defendable comme PFA et extensible vers une solution plus industrielle.

Chapitre 12 - Bibliographie et annexes

12.1 Bibliographie et documentation technique

- React Documentation - <https://react.dev/>
- Next.js Documentation - <https://nextjs.org/docs>
- FastAPI Documentation - <https://fastapi.tiangolo.com/>
- Pydantic Documentation - <https://docs.pydantic.dev/>
- OpenCV Documentation - <https://docs.opencv.org/>
- Tesseract OCR - <https://github.com/tesseract-ocr/tesseract>
- Docling - <https://github.com/docling-project/docling>
- PaddleOCR - <https://github.com/PaddlePaddle/PaddleOCR>
- Ollama Documentation - <https://ollama.com/>
- Qwen Documentation and model cards - <https://qwenlm.github.io/>
- Google Gemini API Documentation - <https://ai.google.dev/>
- SQLite Documentation - <https://www.sqlite.org/docs.html>
- CRISP-DM process model, IBM/SPSS reference materials.

Annexe A - Commandes de lancement

```
python -m pip install -r requirements.txt
ollama pull qwen2.5:7b-instruct
powershell -ExecutionPolicy Bypass -File .\run_api.ps1
powershell -ExecutionPolicy Bypass -File .\run_frontend.ps1

Backend : http://127.0.0.1:8000/api/health
Frontend: http://localhost:3000
```

Annexe B - Variables de configuration principales

Variable	Role	Valeur typique
GEMINI_API_KEY	Cle API cloud.	Secret
GEMINI_MODEL	Modele Gemini.	gemini-2.5-flash
OLLAMA_HOST	Serveur Ollama.	http://127.0.0.1:11434
OLLAMA_MODEL	Modele local.	qwen2.5:7b-instruct
DOCUAI_EXTRACTION_TIMEOUT_SECONDS	Timeout document.	120
OLLAMA_GENERATE_TIMEOUT_SECONDS	Timeout generation.	90
LOCAL_PIPELINE_CONTENT_CHARS	Taille du contenu transmis.	16000
DOCUAI_AUTH_ENABLED	Activation auth.	false en demo
DOCUAI_AUTH_TOKEN_TTL_MINUTES	Duree du token.	480
EXTRACTION_HISTORY_DB_PATH	Base d'historique.	Data/history/extractions.db

Annexe C - Arborescence simplifiee

```
pfaEXTRACT/  
|-- backend/ API FastAPI, auth et orchestration  
|-- frontend/ Next.js, React, TypeScript, Tailwind  
|-- src/  
| |-- extraction/ Extracteurs specialises  
| |-- services/ Routage, pretraitement, IA, historique, PDF  
| |-- models/ Schemas Pydantic  
| `-- web/ Ancienne interface Streamlit  
|-- pipelines/ Pipelines Gemini par famille  
|-- scripts/ Evaluation, QA et fine-tuning  
|-- Data/ Documents, historique et dataset  
|-- outputs/ Resultats d'evaluation et QA  
|-- tests/ Tests pytest  
`-- docs/ Documentation et rapports
```

Annexe D - Scenario de demonstration

1. Lancer Ollama, le backend et le frontend.
2. Ouvrir la page Extractions et importer une image ou un PDF.
3. Choisir Auto et le pipeline local, ou selectionner OCR/Gemini pour comparaison.
4. Lancer l'extraction et observer la redirection vers Documents.
5. Comparer l'aperçu source, les champs, les warnings et le JSON technique.
6. Telecharger le rapport PDF, puis consulter l'historique et le dashboard.
7. Ouvrir Evaluation IA pour expliquer le dataset, les metriques et les limites.

Annexe E - Points a personnaliser avant depot

- Completer le nom de l'etudiant, l'encadrant et l'etablissement sur la couverture.
- Ajouter le logo officiel de l'etablissement si necessaire.
- Insérer des captures d'ecran finales de l'application.
- Remplacer les donnees de presentation du dashboard par les statistiques reelles.
- Ajouter les resultats F1 uniquement apres execution du benchmark complet.
- Relire la terminologie PFA/PFE selon les regles de l'etablissement.